

İSG BİLDİRİM SİSTEMİ

Gün Gün Yapılacaklar

İSG Bildirim Sistemi — 17 İş Günü Uygulama Planı

Amaç:	Projeyi sıfırdan 17 iş gününde bitirmek için günlük görev listesi
Yapı:	5 milestone / 8 sprint / 17 gün
Kullanım:	Her gün ilgili bölümü aç, işleri sırayla yap, gün sonu çıktısını doğrula
Not:	Sprint detay raporları ayrı dosyalarda mevcuttur

Nasıl Kullanılır?

Bu belge, 17 iş gününün her biri için somut görevleri ve o günün sonunda elde etmen gereken çıktıyı listeler. Görevleri yukarıdan aşağıya sırayla yap. Bir günü, 'gün sonu çıktısı' gerçekleşmeden kapatma; gerçekleşmiyorsa eksik kalan işi ertesi güne taşı. Aşağıdaki özet tablo, günlerin milestone ve sprintlere dağılımını gösterir.

Sprint	Günler	Milestone	Odak
Sprint 1	1-2	M1	Temel Altyapı & Şema İskeleti
Sprint 2	3-4	M1	Kimlik Doğrulama & RBAC Çekirdeği
Sprint 3	5-7	M2	Kullanıcı / Rol / Birim Yönetimi
Sprint 4	8-9	M3	İş Kazası Bildirimi & Vücut Haritası
Sprint 5	10-11	M3	Ramak Kala & 5x5 Risk Matrisi
Sprint 6	12-13	M4	İş Akışı, CAPA & Bildirimler
Sprint 7	14-15	M4	Nesne Depolama & Güvenli Dosya Erişimi
Sprint 8	16-17	M5	Denetim, Pano & Cilalama

M1 — Temel Altyapı & Kimlik Doğrulama

Sprint 1 — Temel Altyapı & Şema İskeleti (Gün 1–2)

Projenin çalışabilir iskeletini kurmak: Docker servisleri, Prisma veri modelinin temeli, NestJS ve React+Vite iskeletleri.

Gün 1 — Kurulum & Backend İskeleti

- Git deposu, klasör yapısı (backend/ + client/) ve .gitignore oluştur.
- docker-compose.yml yaz: PostgreSQL (5433), pgAdmin (5050), MinIO (9000/9001) + healthcheck.
- .env / .env.example hazırla (DATABASE_URL, JWT_SECRET, CORS, MinIO, süper admin).
- NestJS iskeletini kur: app.module.ts, main.ts (global /api öneki, CORS, ValidationPipe).
- PrismaModule + PrismaService'i global DI olarak bağla.

✓**Gün sonu çıktısı:** docker compose up ile 3 servis ayakta; backend :3000/api'de boş olarak çalışıyor.

Gün 2 — Veri Şeması & İstemci İskeleti

- schema.prisma: datasource/generator + ilk model iskeleti; prisma migrate ile ilk tabloları oluştur.
- Prisma bağlantısını doğrula (prisma studio).
- Vite + React + TypeScript istemcisini kur; /api proxy'sini ayarla.
- main.tsx (Router + QueryClient) ve App.tsx rota iskeletini yaz.
- styles.css ile global tema/reng paleti; README.md kurulum rehberi.

✓**Gün sonu çıktısı:** İki proje de hatasız derleniyor; client :5173, backend :3000/api; DB'ye bağlanıyor.

Sprint 2 — Kimlik Doğrulama & RBAC Çekirdeği (Gün 3–4)

Kayıt/giriş, JWT oturum, granüler izin (RBAC) modeli ve iki global guard. Süper admin ve roller tohumlanır.

Gün 3 — Kimlik Modeli & Auth Servisi

- schema.prisma'yı genişlet: User, Role, Permission/UserStatus/Rank enum'ları; migrate et.
- auth.service.ts: register (bcrypt hash, status=PENDING), login (parola doğrulama + JWT üretimi), me.
- auth.dto.ts: Register/Login DTO'ları (class-validator kuralları).
- jwt.strategy.ts: token doğrulama + kullanıcıyı DB'den taze yükleme + izin listesi.
- seed.ts: süper admin + 4 sistem rolü (izin setleriyle) + varsayılan birimler.

✓**Gün sonu çıktısı:** Süper admin ile giriş yapıp geçerli JWT alınıyor; seed çalışıyor.

Gün 4 — Global Guard Zinciri & İstemci Oturumu

- JwtAuthGuard + PermissionsGuard'ı APP_GUARD olarak global bağla.
- permissions.decorator (@RequirePermissions) ve current-user.decorator (@CurrentUser) yaz.
- auth.controller: /me, /profile, /password uçları; VERIFIED kapısı ve süper admin kısa devresini test et.
- client api/client.ts: axios instance, Bearer token interceptor, 401'de oturum temizleme.
- AuthContext + LoginPage + RegisterPage + PendingPage: uçtan uca giriş/kayıt/onay-bekleme akışı.

✓**Gün sonu çıktısı:** Yetkisiz uç 403 döner; kayıt→PENDING→giriş akışı arayüzden çalışır.

M2 — Kullanıcı & Yetki Yönetimi

Sprint 3 — Kullanıcı / Rol / Birim Yönetimi (Gün 5–7)

Kullanıcı onay/ret iş akışı, rol/rütbe/birim atama, rol ve izin yönetimi, birim CRUD ve yönetim ekranları.

Gün 5 — Birim & Rol Yönetimi (Backend)

- schema.prisma: Department + UserDepartment (N:M) ekle; migrate et.
- departments.controller: birim CRUD + 3 karakter kod kuralı + audit.
- roles.controller: tüm izinleri listeleme, rol oluştur/güncelle/sil, sistem rolü koruması.
- Uçları @RequirePermissions(ROLE_MANAGE / DEPARTMENT_MANAGE) ile koru.

✓**Gün sonu çıktısı:** Roller ve birimler API'den yönetilebiliyor; sistem rolleri silinmiyor.

Gün 6 — Kullanıcı Onay & Yetkilendirme (Backend)

- users.service: approve/reject (transaction'lı), updateAuthorization, setStatus.
- findAssignable: veri minimizasyonlu (assignableSelect) atanabilir kullanıcı listesi.
- users.controller + user.dto: onay/ret/yetki güncelleme uçları ve doğrulama.
- Onay/ret sonrası NotificationsService ile ilgili kullanıcıya bilgi (altyapı hazırlığı).

✓**Gün sonu çıktısı:** Yönetici bir kullanıcıyı onaylayıp rol/rütbe/birim atayabiliyor (transaction güvenli).

Gün 7 — Yönetim Ekranları (Frontend)

- Layout.tsx: yetkiye göre koşullu menü (hasPermission), dil değiştirici, mobil menü, bildirim zili yuvası.
- AdminUsersPage + UserApprovalModal: liste, onay, rol/birim atama akışı.
- AdminRolesPage: izin matrisi ile rol düzenleme.
- AdminDepartmentsPage: birim CRUD ekranı; uçtan uca elle test.

✓**Gün sonu çıktısı:** Menü izinlere göre değişiyor; tüm yönetim akışları arayüzden çalışıyor.

M3 — Bildirim Çekirdeği

Sprint 4 — İş Kazası Bildirimi & Vücut Haritası (Gün 8–9)

İş kazası bildirimi ve tıklanabilir SVG vücut haritası; Report (tek tablo) ve BodyInjury modelleri.

Gün 8 — Bildirim Modeli & İş Kazası API'si

- schema.prisma: Report (tek tablo, tip ayrımı) + BodyInjury + kaza enum'ları (AccidentType, OutcomeSeverity, BodyPart, InjuryType, BodySide, BodyView); migrate.
- reports.module + reports.service: createAccident, nextReferenceNo (HSE-GG.AA.YYYY-NNN).
- scopeFilter + findAll/findOne: yetki kapsamına göre listeleme ve görünürlük.
- reports.controller + report.dto (accident + BodyInjury alt DTO'ları) + audit.

✓**Gün sonu çıktısı:** İş kazası bildirimi API'den oluşturuluyor; her kayda referans no atanıyor.

Gün 9 — İş Kazası Formu & Vücut Haritası (Frontend)

- BodyMap.tsx: ön/arka SVG insan figürü, bölge tıklama, yaralanma türü/ciddiyeti seçimi.
- AccidentFormPage.tsx: kaza türü, sonuç ciddiyeti, yaralı bilgisi + vücut haritası entegrasyonu.
- Yeni bildirimde REPORT_VIEW_ALL yetkililere in-app duyuru akışını bağla ve test et.

✓**Gün sonu çıktısı:** Kullanıcı vücut haritasında çoklu yaralanma seçip iş kazası bildirebiliyor.

Sprint 5 — Ramak Kala & 5×5 Risk Matrisi (Gün 10–11)

Ramak kala bildirimi ve 5×5 risk matrisi; risk mantığı istemcide önizleme, backend'de otorite olarak.

Gün 10 — Ramak Kala API'si & Risk Mantığı

- schema.prisma: NearMissCategory / HazardCategory / RiskLevel enum'ları + Report near-miss alanları; migrate.
- reports.service: createNearMiss + statik computeRisk (severity×likelihood → LOW/MEDIUM/HIGH/CRITICAL).
- report.dto: CreateNearMiss (tehlike sınıfı, 1–5 skorlar, kök neden, anlık önlem).
- Riskin backend'de yeniden hesaplandığını (istemciye güvenilemediğini) test et.

✓**Gün sonu çıktısı:** Ramak kala bildirimi API'den oluşturuluyor; risk seviyesi sunucuda hesaplanıyor.

Gün 11 — Ramak Kala Formu & 5×5 Matris (Frontend)

- RiskMatrix.tsx: 5×5 ızgara, şiddet×olasılık seçimi, canlı seviye/reng önizlemesi.
- NearMissFormPage.tsx: kategori, tehlike sınıfı, risk skorları, kök neden, anlık önlem alanları.
- İstemci ve sunucu risk sonuçlarının birebir eşleştiğini doğrula; iki form da aynı uca bağlı.

✓**Gün sonu çıktısı:** Kullanıcı ramak kala bildiriyor; risk matrisi renklerle canlı geri bildirim veriyor.

M4 — İş Akışı, CAPA & Dosya

Sprint 6 — İş Akışı, CAPA & Bildirimler (Gün 12–13)

Durum geçiş makinesi, atama, düzeltici faaliyet (CAPA) ve uygulama içi bildirim sistemi; liste/detay ekranları.

Gün 12 — Durum Makinesi & İş Akışı (Backend)

- report-status.ts: izinli geçiş tablosu + canTransition/assertCanTransition + TR etiketler (TEK KAYNAK).
- report-access.ts: canUserViewReport görünürlük kuralını ayrı dosyaya taşı.
- reports.service: updateStatus (transaction + kapatma kuralı: ≥10 karakter + tüm CAPA VERIFIED), assign (otomatik INVESTIGATING), addAction, updateActionStatus, stats.
- schema.prisma: CorrectiveAction + Notification + ReportStatus/ActionStatus/NotificationType; migrate.

✓**Gün sonu çıktısı:** Bildirim durum döngüsü ve CAPA kuralları backend'de eksiksiz çalışıyor.

Gün 13 — Bildirimler & Detay/Liste Ekranları (Frontend)

- notifications.service + controller: notifyByPermission, okundu yönetimi, onay çözümü.
- ReportsListPage: tip/durum/birim filtreleriyle kapsamlı liste.
- ReportDetailPage: durum değiştirme, atama, CAPA ekleme/güncelleme, kapatma.
- NotificationBell + utils/sound: okunmamış rozeti, açılır liste, ses efekti; Layout'a göm.

✓**Gün sonu çıktısı:** Uçtan uca iş akışı: liste → detay → ata → CAPA → kapat; zil sesle uyarıyor.

Sprint 7 — Nesne Depolama & Güvenli Dosya Erişimi (Gün 14–15)

MinIO'ya foto/video yükleme ve yetkilendirilmiş, imzalı sunum; video için Range desteği.

Gün 14 — MinIO Depolama & Yükleme

- storage.service.ts: MinIO client, bucket garantisi (onModuleInit), put (anahtar üretimi), stat/stream/partialStream, presignedUrl.
- storage.module ile StorageService'i global sağla.
- uploads.controller: resim/video filtresi, 100MB limit, MinIO'ya yazma, /api/files/<key> döndürme.
- Yükleme ucunu @RequirePermissions(REPORT_CREATE_*) ile koru ve test et.

✓**Gün sonu çıktısı:** Foto/video MinIO private bucket'a yükleniyor; anahtar geri dönüyor.

Gün 15 — Güvenli Dosya Sunumu (Backend + Frontend)

- files.controller: anahtar biçim doğrulama, ekin bir bildirime ait olduğunu eşleme, report-access ile yetki kontrolü, video Range (206), imzalı URL (≤5 dk, loglanmaz).
- PhotoUpload.tsx: foto/video seçimi + önizleme; ekleri forma iliştiirme (iki formda kullan).
- AuthenticatedMedia.tsx: imzalı URL alıp korumalı medyayı gösterme; ReportDetail'a bağla.

✓**Gün sonu çıktısı:** Ekler yalnızca yetkili kullanıcıya sunuluyor; video ileri/geri sarılabiliyor.

M5 — Gözlemlenebilirlik & Cilalama

Sprint 8 — Denetim, Pano & Cilalama (Gün 16–17)

Kesintisiz denetim kaydı, özet panosu, çoklu dil, PDF çıktı, profil ve son rötuşlar.

Gün 16 — Denetim Kaydı (Audit)

- schema.prisma: AuditLog (actor SetNull, actorName kalıcı, action/entity/meta/ip); migrate.
- audit.service.ts: log (try/catch fail-safe — ana akışı bozmaz) + findAll (filtreli).
- audit.controller: AUDIT_VIEW ile korumalı listeleme.
- Auth/Users/Roles/Reports controller'larına kritik işlem loglamalarını bağla.
- AdminAuditPage: kim-ne-ne zaman kayıtları ekranı.

✓**Gün sonu çıktısı:** Her kritik işlem sessizce loglanıyor; yetkili kullanıcı denetim izini görebiliyor.

Gün 17 — Pano, PDF, Çoklu Dil & Son Cila

- DashboardPage: stats ucundan tip/durum/risk kırılımı + açık faaliyet sayısı.
- PrintableReport: html2pdf ile bildirimi PDF'e çevirme (medyayı 2. sayfaya taşıma).
- i18n (index/tr/en): tüm arayüz metinlerini TR/EN'e bağlama; dil değiştiriciyi test.
- ProfilePage: ad/soyad/telefon + şifre değişikliği.
- Mobil/responsive rötuşlar, uçtan uca duman testi (smoke test), demo provası.

✓**Gün sonu çıktısı:** Ürün demo-hazır: pano, PDF, çoklu dil, profil ve denetim tamamlanmış durumda.

Bitiş — Tamamlanma Kontrol Listesi

17. günün sonunda aşağıdakilerin hepsi ✓ ise proje uçtan uca tamamlanmıştır:

- Docker ile 3 servis (PostgreSQL, pgAdmin, MinIO) tek komutla ayağa kalkıyor.
- Kayıt → onay → rol/birim atama akışı çalışıyor (RBAC).
- İş kazası (vücut haritası) ve ramak kala (5×5 risk) bildirimleri oluşturulabiliyor.
- Durum döngüsü, atama, CAPA ve kapatma kuralları çalışıyor.
- Foto/video güvenli şekilde yükleniyor ve yalnızca yetkiliye sunuluyor.
- Denetim kaydı, pano, PDF çıktı ve TR/EN dil desteği hazır.
- Uçtan uca duman testi (smoke test) ve demo provası yapıldı.